APLICACIÓN DE AHORROS

Sergio Ruiz Ortiz

Fecha:15/02/2026

Curso:1ºDAW

ÍNDICE

1.INTRODUCCIÓN DEL PROYECTO..... 2
2.DESCRIPCIÓN DEL PROYECTO..... 2
3.MODELO ENTIDAD RELACIÓN..... 4
4.MODELO RELACIONAL EN WORKBENCH..... 6
5.CARGA DE DATOS..... 8
6.CAMBIOS CON SET Y ENUM..... 8
7.CONSULTAS SQL..... 9
CONSULTA 1:..... 9
CONSULTA 2:..... 10
CONSULTA 3:..... 11
CONSULTA 4:..... 12
CONSULTA 5:..... 14
CONSULTA 6:..... 15
CONSULTA 7:..... 16
8.VISTAS..... 18
VISTA 1:..... 18
VISTA 2:..... 19
VISTA 3:..... 20
9.PROCEDIMIENTOS,FUNCIONES Y TRIGGERS..... 21
9.1FUNCIONES..... 21
FUNCIÓN 1:..... 21
FUNCIÓN 2:..... 23
9.2PROCEDIMIENTOS..... 27
PROCEDIMIENTO 1:..... 27
PROCEDIMIENTO 2:..... 28
PROCEDIMIENTO 3:..... 31
9.3TRIGGERS..... 33
TRIGGER 1:..... 33
TRIGGER 2:..... 37
10.SUBIDA A GITHUB..... 41
11.CONCLUSIÓN..... 41

1.INTRODUCCIÓN DEL PROYECTO

Este proyecto trata sobre una aplicación para gestionar el dinero personal, pensada para ayudar a las personas a organizar sus ahorros y controlar sus gastos de una forma sencilla e intuitiva.He elegido este proyecto porque considero importante organizar el dinero de forma clara y responsable.

2.DESCRIPCIÓN DEL PROYECTO

Cada usuario puede registrarse en la aplicación y se guardan sus datos personales y de acceso, como su identificador, nombre, apellidos, fecha de nacimiento, nombre de usuario, contraseña, correo electrónico y el país en el que vive. El usuario es el elemento principal del sistema, ya que es quien realiza todas las acciones dentro de la aplicación.

Una vez registrado, el usuario puede crear distintas metas de ahorro, por ejemplo ahorrar para un viaje o para una compra importante. De cada meta se guarda su identificador, el nombre de la meta, el estado en el que se encuentra, la prioridad, la cantidad total que se desea ahorrar y la cantidad que se ha conseguido hasta el momento. Además, se registran las fechas de inicio y de fin de cada meta.

Para gestionar el dinero dentro de las metas de ahorro, el usuario puede realizar transacciones, que permiten añadir o retirar dinero. Cada transacción queda asociada a un usuario y a una meta concreta, y se guarda el número de transacción, la cantidad, el tipo de movimiento (ingreso o retirada) y la fecha en la que se realiza.

Además, el usuario puede crear distintas listas de presupuesto, que sirven para establecer límites de gasto. Por ejemplo, se puede definir un presupuesto para comida, ocio o compras. De cada lista se guarda su identificador, el nombre, una descripción y la cantidad máxima que se puede gastar .

Las listas de presupuesto se organizan mediante categorías, que permiten identificar fácilmente el tipo de gasto. De cada categoría se guarda su identificador, el nombre, una breve descripción y el color del icono que la representa. La aplicación también

incluye un sistema de notificaciones, que informa al usuario sobre distintos eventos importantes, como el estado de una meta de ahorro o avisos relacionados con sus presupuestos.

De cada notificación se guarda su identificador, el motivo, el título, el mensaje y su prioridad, así como la fecha en la que se recibe.

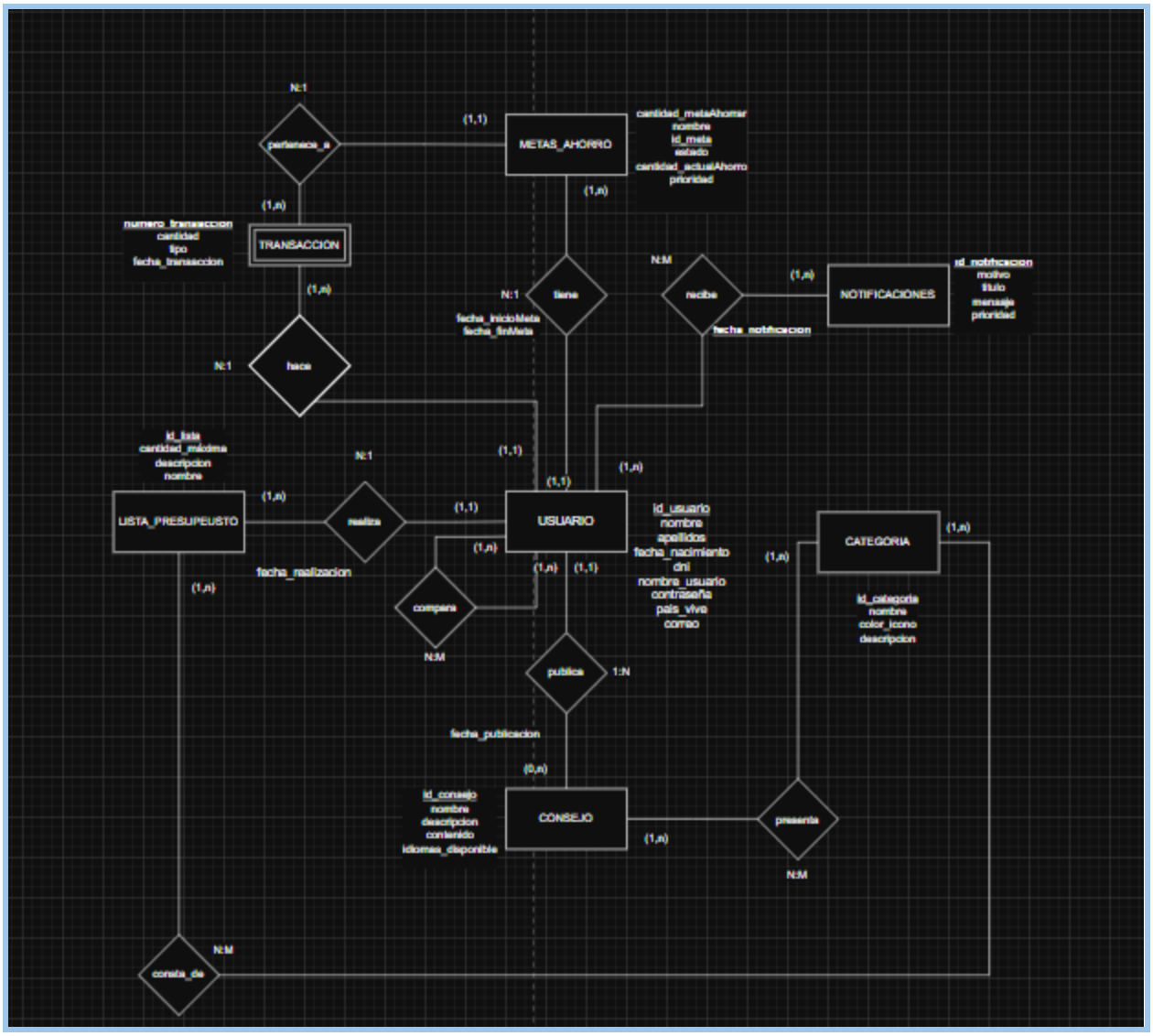
Por otro lado, los usuarios pueden publicar consejos relacionados con la gestión del dinero, con el objetivo de ayudar a otros usuarios. De cada consejo se guarda su identificador, el nombre, una descripción y el contenido, además de la fecha en la que se publica.

Por último, la aplicación permite que los usuarios se comparen entre sí, de manera que puedan ver su situación económica en relación con otros usuarios, fomentando un la competencia entre usuarios, lo que los va a llevar a tomárselo más en serio.

3.MODELO ENTIDAD RELACIÓN

Para que la aplicación funcione bien, lo primero que he hecho es organizar cómo se va a guardar toda la información. El centro de todo es el usuario, porque es quien hace todo en la app. A partir de ahí, he creado varias entidades que se conectan entre sí:

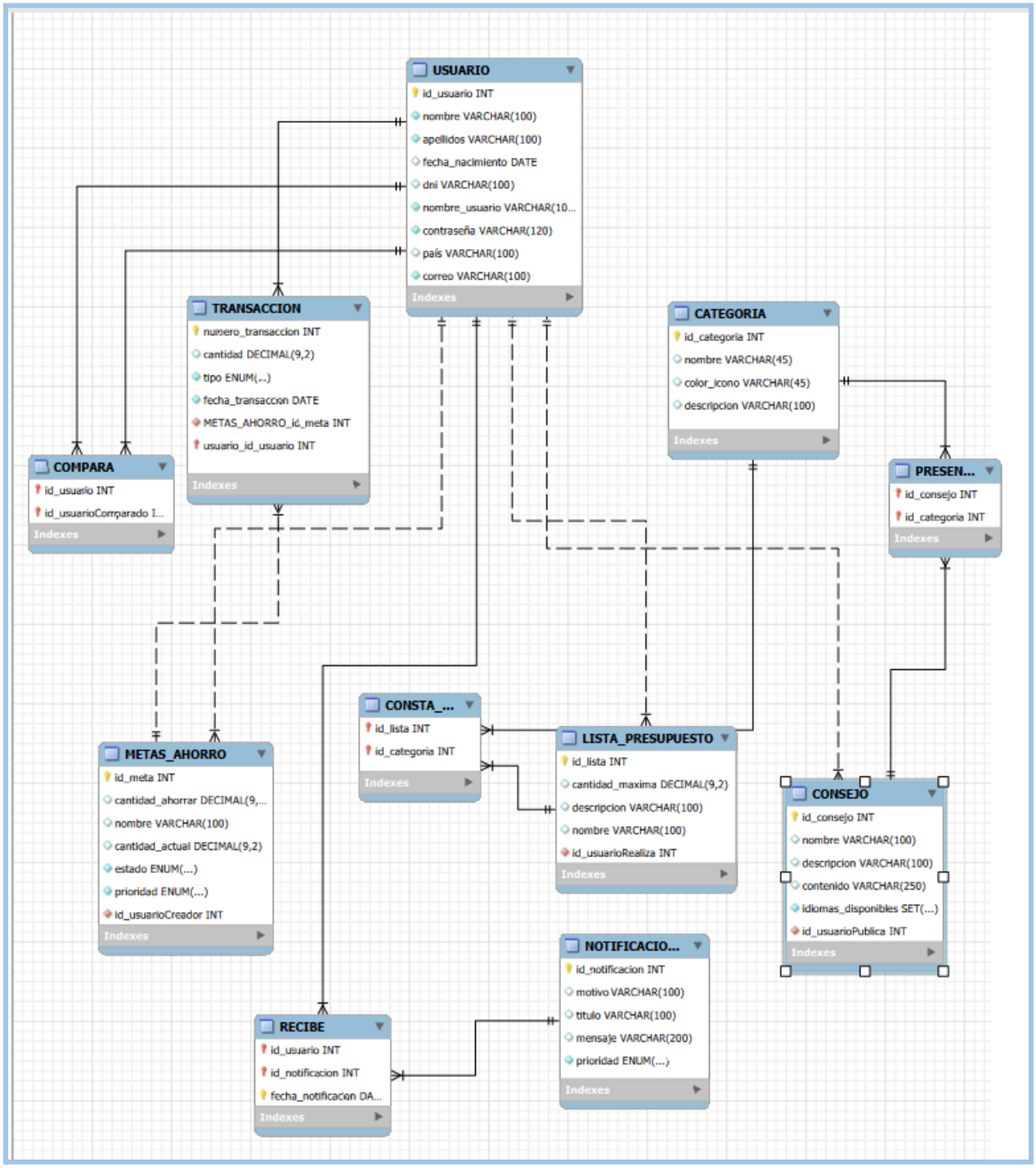
- **Usuario**: Aquí guardo los datos básicos de la persona, como su nombre, correo y país. Sin el usuario, nada de lo demás tendría sentido.
- **Metas de Ahorro**: Son los objetivos que se propone cada uno, cómo ahorrar para un viaje. Guardo cuánto quieren conseguir y qué prioridad tiene cada meta.
- **Transacciones**: Cada vez que alguien mete o saca dinero de una meta, se registra aquí. Así sabemos exactamente cuánto llevan ahorrado en cada momento.
- **Listas de Presupuesto**: Sirven para poner límites y no gastar de más en cosas como comida u ocio.
- **Categorías**: Las uso para organizar esos presupuestos. Por ejemplo, para saber si un gasto es de ocio o de comida.
- **Notificaciones**: Son los avisos que le llegan al usuario para decirle cómo van sus ahorros o si se está pasando de su presupuesto o cualquier otro aviso importante.
- **Consejos**: Es una parte donde los usuarios pueden escribir recomendaciones para ayudar a otros a ahorrar dinero.


N:1
pertenece_a
(1,1)
METAS_AHORRO
cantidad_metaAhorrar
nombre
id_meta
estado
cantidad_actualAhorro
prioridad
(1,n)
(1,n)
numero_transaccion
cantidad
tipo
fecha_transaccion
TRANSACCIÓN
(1,n)
N:1
tiene
fecha_inicioMeta
fecha_finMeta
N:M
recibe
(1,n)
NOTIFICACIONES
id_notificacion
motivo
titulo
mensaje
prioridad
fecha_notificacion
N:1
hace
id_lista
cantidad_máxima
descripcion
nombre
N:1
(1,1)
(1,n)
(1,1)
(1,n)
LISTA_PRESUPUESTO
(1,n)
realiza
(1,1)
USUARIO
id_usuario
nombre
apellidos
fecha_nacimiento
dni
nombre_usuario
contraseña
pais_vive
correo
CATEGORIA
(1,n)
(1,n)
fecha_realización
(1,n)
(1,n)
(1,1)
compara
N:M
publica
1:N
id_categoria
nombre
color_icono
descripcion
fecha_publicacion
(0,n)
id_consejo
nombre
descripcion
contenido
idiomas_disponible
CONSEJO
(1,n)
presenta
N:M
N:M
consta_de

4.MODELO RELACIONAL EN WORKBENCH

Cuando ya tenía el esquema claro, me pasé a MySQL Workbench para montar la base de datos de verdad. Aquí es donde creé las tablas que luego se usan en el servidor, intentando que todo fuera lo más fiel posible al modelo Entidad-Relación que hice al principio.

En el Workbench fui copiando lo que ya tenía hecho, pero aquí ya tenía que elegir bien el tipo de dato de cada atributo para que no me diese fallos. También hay que fijarse en las relaciones: las líneas continuas son para las relaciones fuertes y las discontinuas para las débiles, para que el diagrama se entienda perfectamente y no dé errores luego al conectarlas.


USUARIO
id_usuario INT
nombre VARCHAR(100)
apellidos VARCHAR(100)
fecha_nacimiento DATE
dni VARCHAR(100)
nombre_usuario VARCHAR(10...
contraseña VARCHAR(120)
país VARCHAR(100)
correo VARCHAR(100)
Indexes
TRANSACCION
numero_transaccion INT
cantidad DECIMAL(9,2)
tipo ENUM(...)
fecha_transaccion DATE
METAS_AHORRO_id_meta INT
usuario_id_usuario INT
Indexes
CATEGORIA
id_categoria INT
nombre VARCHAR(45)
color_icono VARCHAR(45)
descripcion VARCHAR(100)
Indexes
COMPARA
id_usuario INT
id_usuarioComparado I...
Indexes
PRESEN...
id_consejo INT
id_categoria INT
Indexes
CONSTA_...
id_lista INT
id_categoria INT
Indexes
LISTA_PRESUPUESTO
id_lista INT
cantidad_maxima DECIMAL(9,2)
descripcion VARCHAR(100)
nombre VARCHAR(100)
id_usuarioRealiza INT
Indexes
METAS_AHORRO
id_meta INT
cantidad_ahorrar DECIMAL(9,...
nombre VARCHAR(100)
cantidad_actual DECIMAL(9,2)
estado ENUM(...)
prioridad ENUM(...)
id_usuarioCreador INT
Indexes
CONSEJO
id_consejo INT
nombre VARCHAR(100)
descripcion VARCHAR(100)
contenido VARCHAR(250)
idiomas_disponibles SET(...)
id_usuarioPublica INT
Indexes
NOTIFICACIO...
id_notificacion INT
motivo VARCHAR(100)
titulo VARCHAR(100)
mensaje VARCHAR(200)
prioridad ENUM(...)
Indexes
RECIBE
id_usuario INT
id_notificacion INT
fecha_notificacion DA...
Indexes

5.CARGA DE DATOS

Para la carga de datos usé una web que se llama Mackaroo. Básicamente, le dije qué columnas quería (nombres, fechas, cantidades, lo que sea) y la página me generó un montón de filas de datos al momento. Luego, esos datos los cargué directamente en mi sistema.

Sin embargo, para la tabla de Metas de Ahorro, no quise inventar la cantidadActual con Mackaroo porque corría el riesgo de que los números no cuadrasen con los movimientos reales. Lo que hice fue usar las funciones y procedimientos que ya había programado. Así, en lugar de poner un número al azar, el sistema calculaba automáticamente cuánto dinero llevaba ahorrado cada usuario sumando sus ingresos y restando sus gastos de la tabla de transacciones. De esta forma, me aseguré de que los datos de las metas fueran totalmente reales y estuvieran siempre sincronizados con la cantidad que tenía el usuario en su meta de ahorro..

6.CAMBIOS CON SET Y ENUM

En esta parte del proyecto, decidí hacer unos ajustes en las tablas para que la base de datos fuera más profesional y no permitiera errores al meter los datos. En lugar de dejar que se pudiera escribir cualquier cosa, usé los tipos ENUM y SET para limitar las opciones:

- **ENUM**: Lo he aplicado en cosas como el estado de las metas o la prioridad. Así, solo se puede elegir entre opciones fijas como activo, pendiente o inactivo lo que puede servir para evitar que alguien escriba mal una palabra y luego las consultas no funcionen.
- **SET:** Este lo he usado para los Idiomas disponibles en los consejos, agregando la columna directamente a esa tabla. La diferencia con el ENUM es que aquí un mismo consejo puede estar en varios idiomas a la vez, como español e inglés, y el tipo SET me deja marcar varias opciones en una sola columna.

```sql
ALTER TABLE METAS_AHORRO MODIFY COLUMN estado ENUM('activo', 'pendiente', 'inactivo') NOT NULL DEFAULT 'inactivo';

ALTER TABLE METAS_AHORRO MODIFY COLUMN prioridad ENUM('baja','media','alta') not null default 'baja';

ALTER TABLE NOTIFICACIONES modify column prioridad ENUM('baja','media','alta') not null default 'baja';

ALTER TABLE TRANSACCION  modify column tipo ENUM('ingreso','retirada') not null;

alter table PRESENTA CHANGE id_coonsejo id_consejo INT;

ALTER TABLE CONSEJO CHANGE id_coonsejo id_consejo INT;

ALTER TABLE CONSEJO
ADD COLUMN idiomas_disponibles SET('es', 'en', 'fr', 'pt', 'de', 'it')
NOT NULL DEFAULT 'es' AFTER contenido;
```

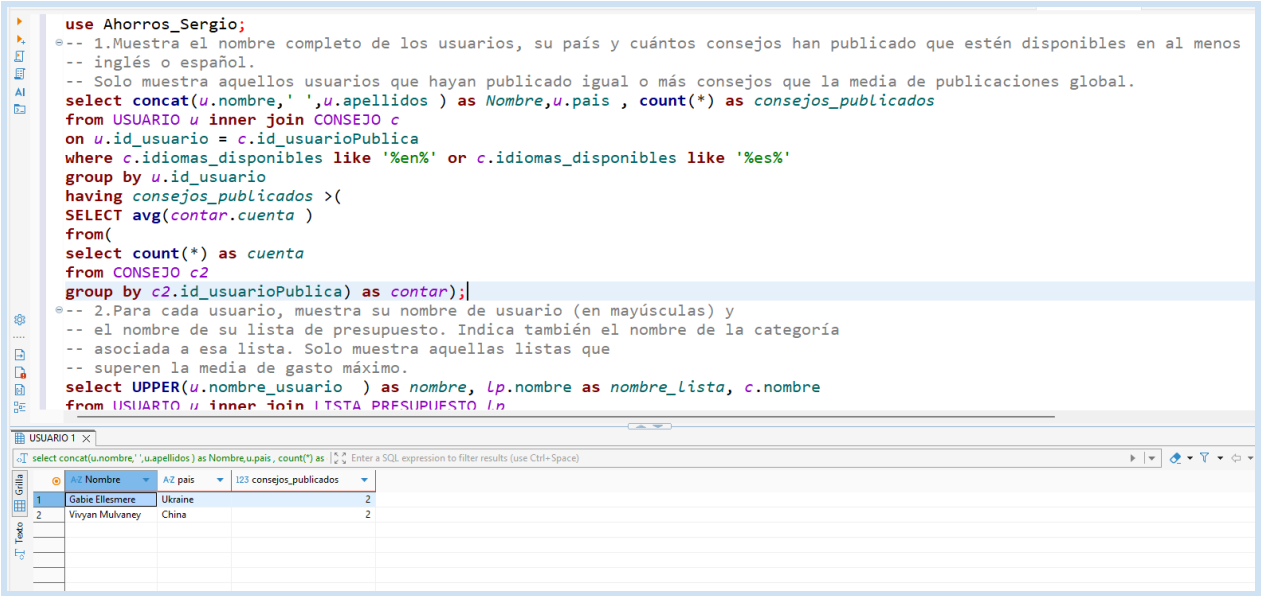
7.CONSULTAS SQL

CONSULTA 1:

1.Muestra el nombre completo de los usuarios, su país y cuántos consejos han publicado siempre que estos consejos están en inglés o español. Solo mostraremos aquellos usuarios que hayan publicado igual o más consejos que la media de publicaciones global.

```sql
-- 1.Muestra el nombre completo de los usuarios, su país y cuántos consejos han publicado que estén disponibles en al menos
-- inglés o español.
-- Solo muestra aquellos usuarios que hayan publicado igual o más consejos que la media de publicaciones global.
select concat(u.nombre,' ',u.apellidos ) as Nombre,u.pais , count(*) as consejos_publicados
from USUARIO u inner join CONSEJO c
on u.id_usuario = c.id_usuarioPublica
where c.idiomas_disponibles like '%en%' or c.idiomas_disponibles like '%es%'
```

```
group by u.id_usuario
having consejos_publicados >(
SELECT avg(contar.cuenta )
from(
select count(*) as cuenta
from CONSEJO c2
group by c2.id_usuarioPublica) as contar);
```

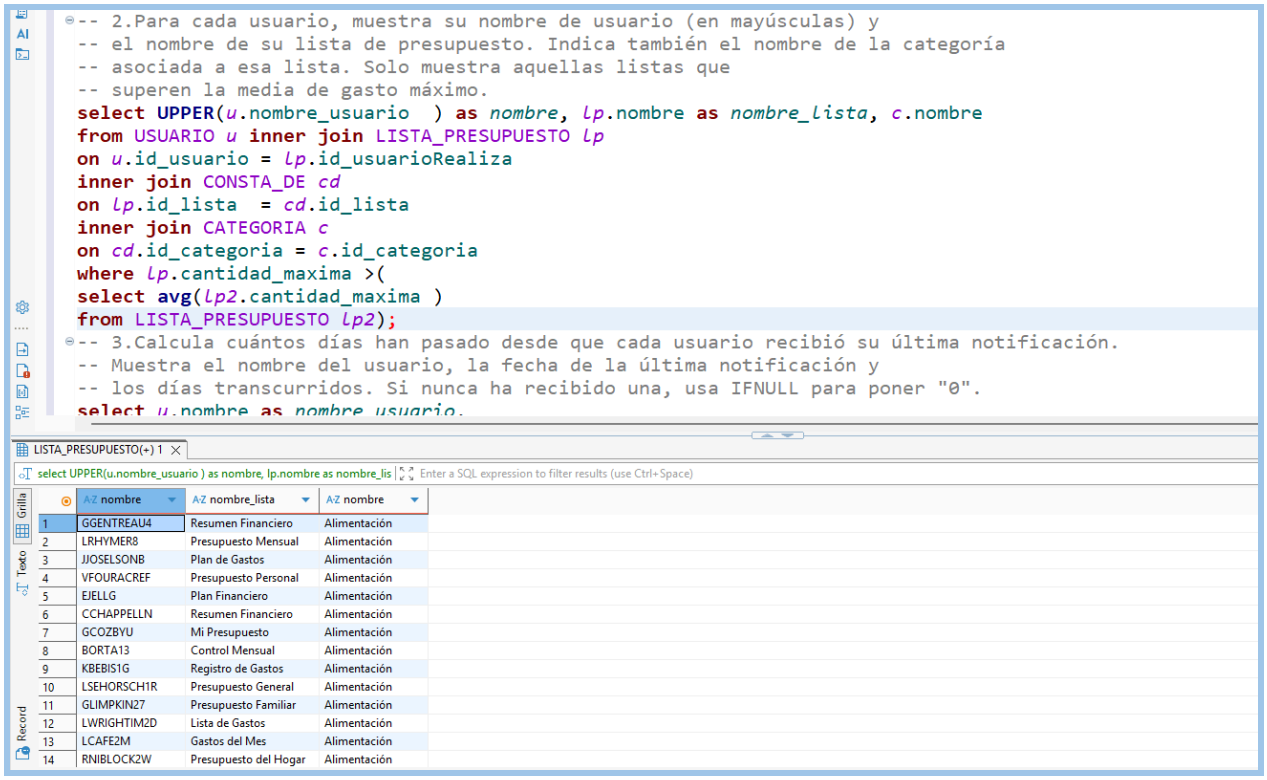


CONSULTA 2:

2.Para cada usuario, muestra su nombre de usuario en mayúsculas y el nombre de su lista de presupuesto. Indica también el nombre de la categoría asociada a esa lista. Solo muestra aquellas listas que superen la media de gasto máximo.

```
-- 2.Para cada usuario, muestra su nombre de usuario (en mayúsculas) y
-- el nombre de su lista de presupuesto. Indica también el nombre de la categoría
-- asociada a esa lista. Solo muestra aquellas listas que
-- superen la media de gasto máximo.
select UPPER(u.nombre_usuario ) as nombre, lp.nombre as nombre_lista, c.nombre
from USUARIO u inner join LISTA_PRESUPUESTO lp
on u.id_usuario = lp.id_usuarioRealiza
```

```
inner join CONSTA_DE cd
on lp.id_lista  = cd.id_lista
inner join CATEGORIA c
on cd.id_categoria = c.id_categoria
where lp.cantidad_maxima >(
select avg(lp2.cantidad_maxima )
from LISTA_PRESUPUESTO lp2);
```



```
-- 2.Para cada usuario, muestra su nombre de usuario (en mayúsculas) y
-- el nombre de su lista de presupuesto. Indica también el nombre de la categoría
-- asociada a esa lista. Solo muestra aquellas listas que
-- superen la media de gasto máximo.
select UPPER(u.nombre_usuario  ) as nombre, lp.nombre as nombre_lista, c.nombre
from USUARIO u inner join LISTA_PRESUPUESTO lp
on u.id_usuario = lp.id_usuarioRealiza
inner join CONSTA_DE cd
on lp.id_lista  = cd.id_lista
inner join CATEGORIA c
on cd.id_categoria = c.id_categoria
where lp.cantidad_maxima >(
select avg(lp2.cantidad_maxima )
from LISTA_PRESUPUESTO lp2);
-- 3.Calcula cuántos días han pasado desde que cada usuario recibió su última notificación.
-- Muestra el nombre del usuario, la fecha de la última notificación y
-- los días transcurridos. Si nunca ha recibido una, usa IFNULL para poner "0".
select u.nombre as nombre_usuario,
```

LISTA_PRESUPUESTO(+) 1

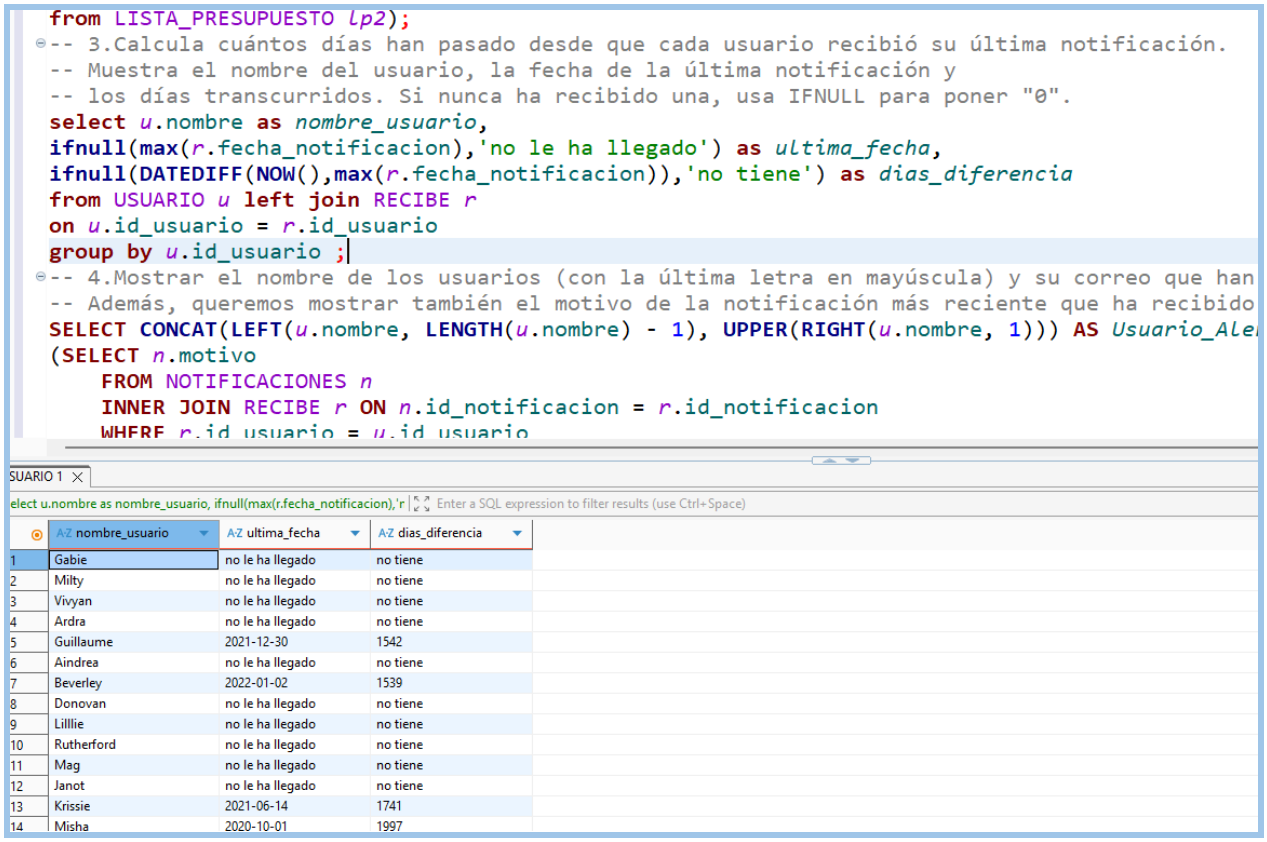
select UPPER(u.nombre_usuario) as nombre, lp.nombre as nombre_lis | Enter a SQL expression to filter results (use Ctrl+Space)

	nombre	nombre_lista	nombre
1	GGENTREAU4	Resumen Financiero	Alimentación
2	LRHYMER8	Presupuesto Mensual	Alimentación
3	JJOSELSONB	Plan de Gastos	Alimentación
4	VFOURACREF	Presupuesto Personal	Alimentación
5	EJELLG	Plan Financiero	Alimentación
6	CCHAPPELLN	Resumen Financiero	Alimentación
7	GCOZBYU	Mi Presupuesto	Alimentación
8	BORTA13	Control Mensual	Alimentación
9	KBEBIS1G	Registro de Gastos	Alimentación
10	LSEHORSCH1R	Presupuesto General	Alimentación
11	GLIMPKIN27	Presupuesto Familiar	Alimentación
12	LWRIGHTIM2D	Lista de Gastos	Alimentación
13	LCAFE2M	Gastos del Mes	Alimentación
14	RNIBLOCK2W	Presupuesto del Hogar	Alimentación

CONSULTA 3:

3.Calcula cuántos días han pasado desde que cada usuario recibió su última notificación.Muestra el nombre del usuario, la fecha de la última notificación y los días transcurridos. Si nunca ha recibido una, usamos IFNULL para poner "no ha llegado" en la columna de las fechas, y no tienen en la de los días transcurridos.

```
-- 3.Calcula cuántos días han pasado desde que cada usuario recibió su última notificación.
-- Muestra el nombre del usuario, la fecha de la última notificación y
-- los días transcurridos. Si nunca ha recibido una, usa IFNULL para poner "0".
select u.nombre as nombre_usuario,
ifnull(max(r.fecha_notificacion),'no le ha llegado') as ultima_fecha,
ifnull(DATEDIFF(NOW(),max(r.fecha_notificacion)),'no tiene') as dias_diferencia
from USUARIO u left join RECIBE r
on u.id_usuario = r.id_usuario
group by u.id_usuario ;
```



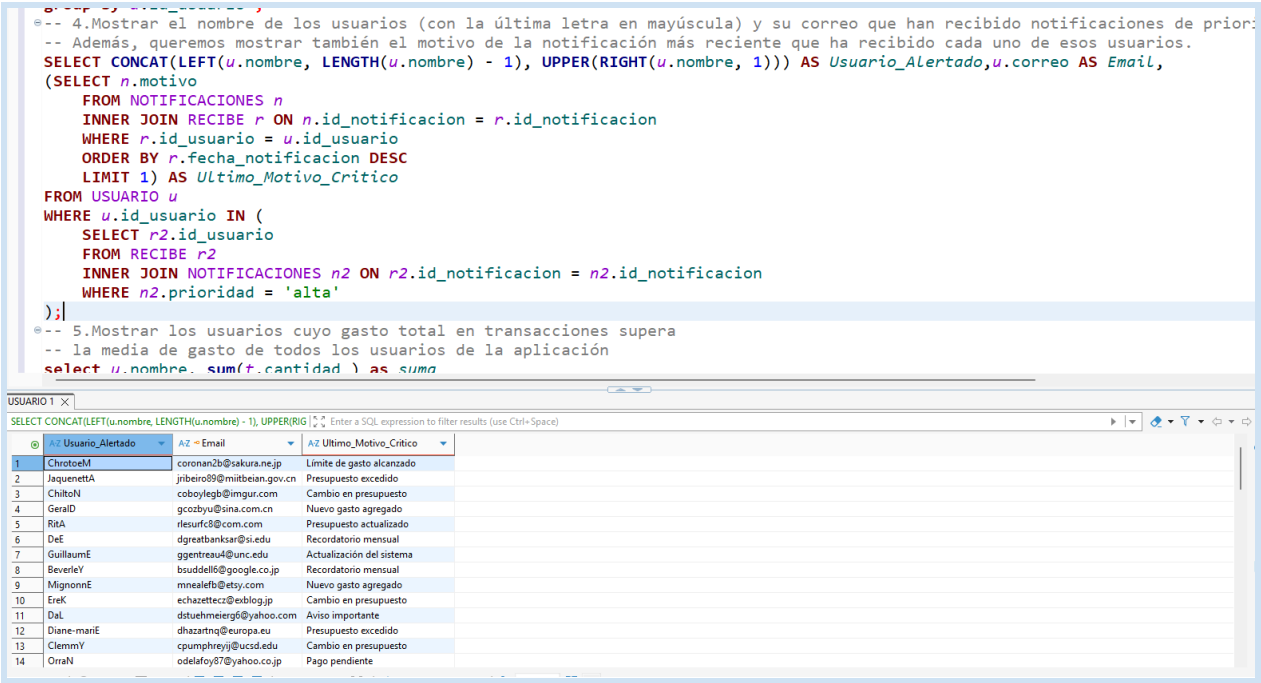
	nombre_usuario	ultima_fecha	dias_diferencia
1	Gabie	no le ha llegado	no tiene
2	Milty	no le ha llegado	no tiene
3	Vivyan	no le ha llegado	no tiene
4	Ardra	no le ha llegado	no tiene
5	Guillaume	2021-12-30	1542
6	Aindrea	no le ha llegado	no tiene
7	Beverley	2022-01-02	1539
8	Donovan	no le ha llegado	no tiene
9	Lilllie	no le ha llegado	no tiene
10	Rutherford	no le ha llegado	no tiene
11	Mag	no le ha llegado	no tiene
12	Janot	no le ha llegado	no tiene
13	Krissie	2021-06-14	1741
14	Misha	2020-10-01	1997

CONSULTA 4:

4 Mostrar el nombre de los usuarios (con la última letra en mayúscula) y su correo que han recibido notificaciones de prioridad 'alta'. Mediante una subconsulta, mostrar

también el motivo de la notificación más reciente que ha recibido cada uno de esos usuarios.

```
-- 4.Mostrar el nombre de los usuarios (con la última letra en mayúscula) y su correo
que han recibido notificaciones de prioridad 'alta'.
-- Además, queremos mostrar también el motivo de la notificación más reciente que ha
recibido cada uno de esos usuarios.
SELECT CONCAT(LEFT(u.nombre, LENGTH(u.nombre) - 1),
UPPER(RIGHT(u.nombre, 1))) AS Usuario_Alertado,u.correo AS Email,
(SELECT n.motivo
        FROM NOTIFICACIONES n
        INNER JOIN RECIBE r ON n.id_notificacion = r.id_notificacion
        WHERE r.id_usuario = u.id_usuario
        ORDER BY r.fecha_notificacion DESC
        LIMIT 1) AS Ultimo_Motivo_Critico
FROM USUARIO u
WHERE u.id_usuario IN (
        SELECT r2.id_usuario
        FROM RECIBE r2
        INNER JOIN NOTIFICACIONES n2 ON r2.id_notificacion = n2.id_notificacion
        WHERE n2.prioridad = 'alta'
);
```



```
-- 4.Mostrar el nombre de los usuarios (con la última letra en mayúscula) y su correo que han recibido notificaciones de prior
-- Además, queremos mostrar también el motivo de la notificación más reciente que ha recibido cada uno de esos usuarios.
SELECT CONCAT(LEFT(u.nombre, LENGTH(u.nombre) - 1), UPPER(RIGHT(u.nombre, 1))) AS Usuario_Alertado,u.correo AS Email,
(SELECT n.motivo
    FROM NOTIFICACIONES n
    INNER JOIN RECIBE r ON n.id_notificacion = r.id_notificacion
    WHERE r.id_usuario = u.id_usuario
    ORDER BY r.fecha_notificacion DESC
    LIMIT 1) AS Ultimo_Motivo_Critico
FROM USUARIO u
WHERE u.id_usuario IN (
    SELECT r2.id_usuario
    FROM RECIBE r2
    INNER JOIN NOTIFICACIONES n2 ON r2.id_notificacion = n2.id_notificacion
    WHERE n2.prioridad = 'alta'
);
-- 5.Mostrar los usuarios cuyo gasto total en transacciones supera
-- la media de gasto de todos los usuarios de la aplicación
select u.nombre, sum(t.cantidad ) as suma
```

USUARIO 1

SELECT CONCAT(LEFT(u.nombre, LENGTH(u.nombre) - 1), UPPER(RIG

	Usuario_Alertado	Email	Ultimo_Motivo_Critico
1	ChrotoeM	coronan2b@sakura.ne.jp	Límite de gasto alcanzado
2	JaquenettA	jribeiro89@miitbeian.gov.cn	Presupuesto excedido
3	ChiltoN	coboylegb@imgur.com	Cambio en presupuesto
4	GeralD	gcozbyu@sina.com.cn	Nuevo gasto agregado
5	RitA	rlesuirfc8@com.com	Presupuesto actualizado
6	DeE	dgreatbanksar@si.edu	Recordatorio mensual
7	GuillaumE	ggentreau4@unc.edu	Actualización del sistema
8	BeverleY	bsuddell6@google.co.jp	Recordatorio mensual
9	MignonnE	mnealefb@etsy.com	Nuevo gasto agregado
10	EreK	echazettecz@exblog.jp	Cambio en presupuesto
11	DaL	dstuehmeierg6@yahoo.com	Aviso importante
12	Diane-marIE	dhazartnq@europa.eu	Presupuesto excedido
13	ClemmY	cpumphreyij@ucsd.edu	Cambio en presupuesto
14	OrraN	odelafoy87@yahoo.co.jp	Pago pendiente

CONSULTA 5:

5.Mostrar los usuarios cuyo gasto total en transacciones supera la media de gasto de todos los usuarios de la aplicación

```
-- 5.Mostrar los usuarios cuyo gasto total en transacciones supera
-- la media de gasto de todos los usuarios de la aplicación
select u.nombre, sum(t.cantidad ) as suma
from USUARIO u inner join TRANSACCION t
on u.id_usuario = t.id_usuario
group by u.id_usuario
having suma >(
select avg(cantidad_empelado.cantidad )
from(
select sum(t2.cantidad) as cantidad
from USUARIO u2 inner join TRANSACCION t2
on u2.id_usuario = t2.id_usuario
group by u2.id_usuario) as cantidad_empelado);
```

```
);
-- 5.Mostrar los usuarios cuyo gasto total en transacciones supera
-- la media de gasto de todos los usuarios de la aplicación
select u.nombre, sum(t.cantidad ) as suma
from USUARIO u inner join TRANSACCION t
on u.id_usuario = t.id_usuario
group by u.id_usuario
having suma >(
select avg(cantidad_empelado.cantidad )
from(
select sum(t2.cantidad) as cantidad
from USUARIO u2 inner join TRANSACCION t2
on u2.id_usuario = t2.id_usuario
group by u2.id_usuario) as cantidad_empelado);
-- 6.Encontrar usuarios que se han registrado pero no han creado ninguna lista de
select u.nombre
```

USUARIO 1

select u.nombre, sum(t.cantidad) as suma from USUARIO u inner join | Enter a SQL expression to filter results (use Ctrl+Space)

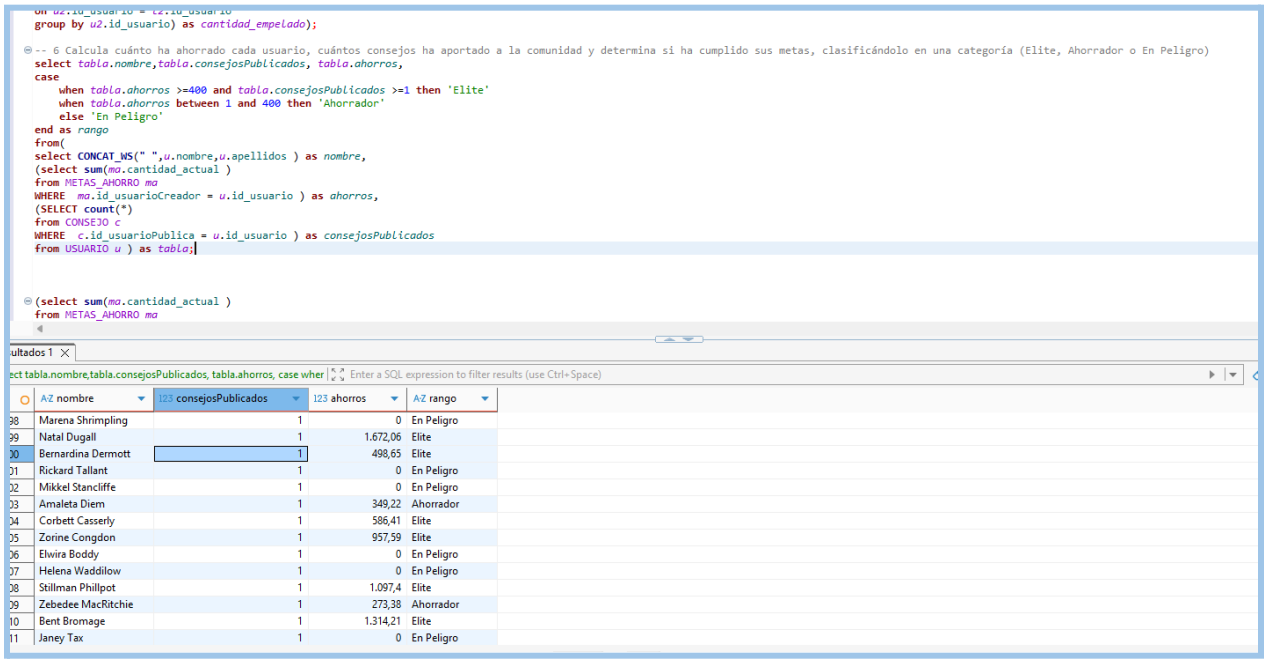
	nombre	suma
1	Gabie	2.355,15
2	Milty	2.382,87
3	Beverley	1.913,87
4	Mag	1.823,88
5	Krissie	2.069,45
6	Misha	2.679,63
7	Andi	2.065,43
8	Virgil	1.523,16
9	Magdaia	2.092,75
10	Hartley	3.184,12
11	Henriette	1.579,19
12	Perkin	1.488,08
13	Taryn	1.463,65
14	Bria	1.690,08

CONSULTA 6:

6. Calcula cuánto ha ahorrado cada usuario, cuántos consejos ha aportado a la comunidad y determina si ha cumplido sus metas, clasificándose en una categoría (Elite, Ahorrador o En Peligro)

```
-- 6 Calcula cuánto ha ahorrado cada usuario, cuántos consejos ha aportado a la
comunidad y determina si ha cumplido sus metas, clasificándolo en una categoría
(Elite, Ahorrador o En Peligro)
select tabla.nombre,tabla.consejosPublicados, tabla.ahorros,
case
        when tabla.ahorros >=400 and tabla.consejosPublicados >=1 then 'Elite'
        when tabla.ahorros between 1 and 400 then 'Ahorrador'
        else 'En Peligro'
```

```
end as rango
from(
select CONCAT_WS(" ",u.nombre,u.apellidos ) as nombre,
(select sum(ma.cantidad_actual )
from METAS_AHORRO ma
where  ma.id_usuarioCreador = u.id_usuario ) as ahorros,
(select count(*)
from CONSEJO c
where  c.id_usuarioPublica = u.id_usuario ) as consejosPublicados
from USUARIO u ) as tabla;
```

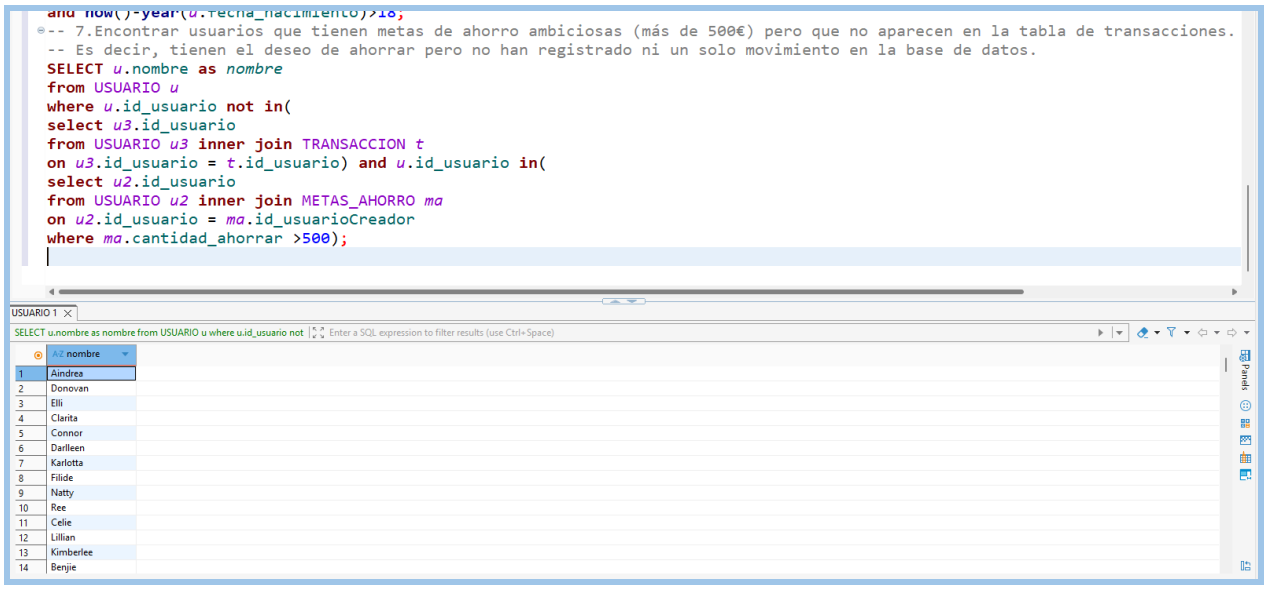


CONSULTA 7:

7.Encontrar usuarios que tienen metas de ahorro ambiciosas (más de 500€) pero que no aparecen en la tabla de transacciones. Es decir, tienen el deseo de ahorrar pero no han registrado ni un solo movimiento en la base de datos.

-- 7.Encontrar usuarios que tienen metas de ahorro ambiciosas (más de 500€) pero que no aparecen en la tabla de transacciones.
-- Es decir, tienen el deseo de ahorrar pero no han registrado ni un solo movimiento en la base de datos.

```
SELECT u.nombre as nombre
from USUARIO u
where u.id_usuario not in(
select u3.id_usuario
from USUARIO u3 inner join TRANSACCION t
on u3.id_usuario = t.id_usuario) and u.id_usuario in(
select u2.id_usuario
from USUARIO u2 inner join METAS_AHORRO ma
on u2.id_usuario = ma.id_usuarioCreador
where ma.cantidad_ahorrar >500);
```

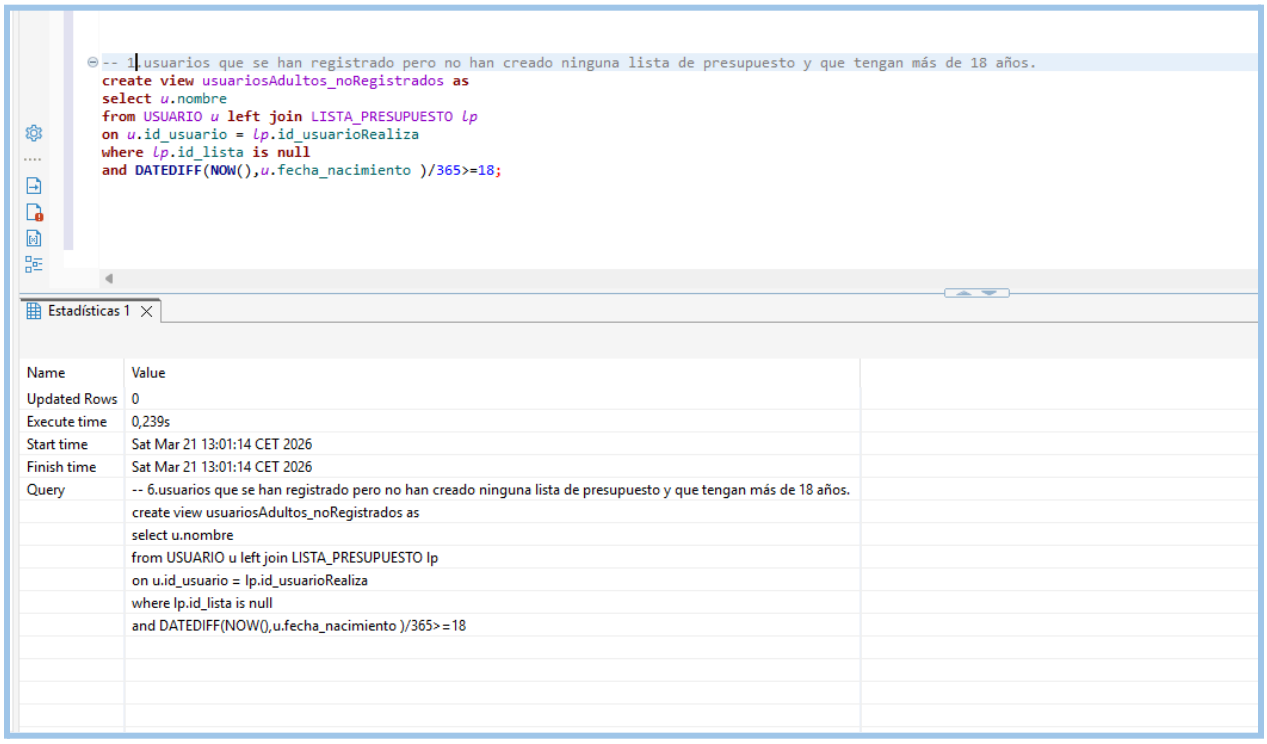


8.VISTAS

VISTA 1:

1.usuarios que se han registrado pero no han creado ninguna lista de presupuesto y que tengan más de 18 años.

```
-- 1.usuarios que se han registrado pero no han creado ninguna lista de presupuesto y que tengan más de 18 años.
create view usuariosAdultos_noRegistrados as
select u.nombre
from USUARIO u left join LISTA_PRESUPUESTO lp
on u.id_usuario = lp.id_usuarioRealiza
where lp.id_lista is null
and DATEDIFF(NOW(),u.fecha_nacimiento )/365>=18;
```



VISTA 2:

2.Usuarios que tienen metas de ahorro ambiciosas (más de 500€) pero que no aparecen en la tabla de transacciones. Es decir, tienen el deseo de ahorrar pero no han registrado ni un solo movimiento en la base de datos.

```
-- 2.Usuarios que tienen metas de ahorro ambiciosas (más de 500€) pero que no
aparecen en la tabla de transacciones.
-- Es decir, tienen el deseo de ahorrar pero no han registrado ni un solo movimiento en
la base de datos.
create view ambiciosos as
SELECT u.nombre as nombre
from USUARIO u
where u.id_usuario not in(
select u3.id_usuario
from USUARIO u3 inner join TRANSACCION t
on u3.id_usuario = t.id_usuario) and u.id_usuario in(
select u2.id_usuario
from USUARIO u2 inner join METAS_AHORRO ma
on u2.id_usuario = ma.id_usuarioCreador
where ma.cantidad_ahorrar >500);
```

```
-- 2.Usuarios que tienen metas de ahorro ambiciosas (más de 500€) pero que no aparecen en la tabla de transacciones.
-- Es decir, tienen el deseo de ahorrar pero no han registrado ni un solo movimiento en la base de datos.
create view ambiciosos as
SELECT u.nombre as nombre
from USUARIO u
where u.id_usuario not in(
select u3.id_usuario
from USUARIO u3 inner join TRANSACCION t
on u3.id_usuario = t.id_usuario) and u.id_usuario in(
select u2.id_usuario
from USUARIO u2 inner join METAS_AHORRO ma
on u2.id_usuario = ma.id_usuarioCreador
where ma.cantidad_ahorrar >500);
```

adísticas 1

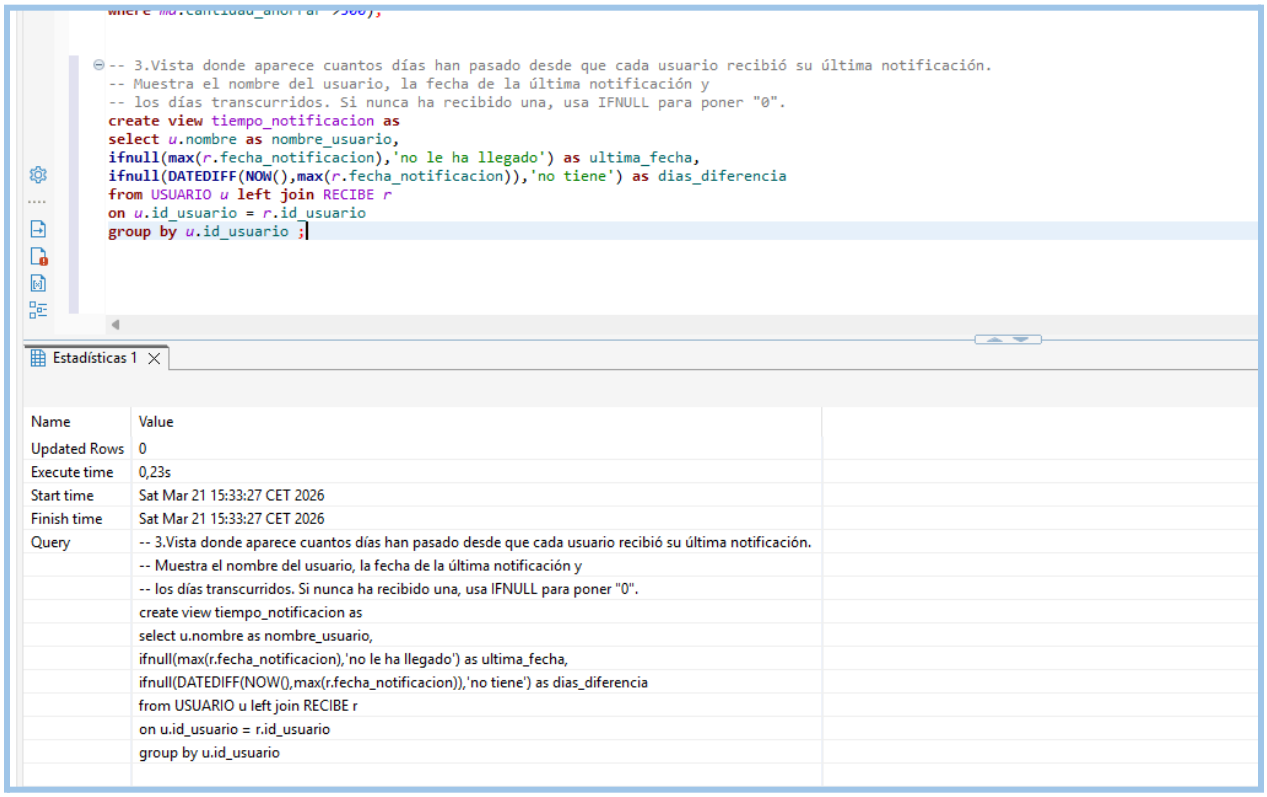
	Value
ed Rows	0
te time	0,235s
ime	Sat Mar 21 13:07:09 CET 2026
time	Sat Mar 21 13:07:10 CET 2026
	-- 2.Usuarios que tienen metas de ahorro ambiciosas (más de 500€) pero que no aparecen en la tabla de transacciones.
	-- Es decir, tienen el deseo de ahorrar pero no han registrado ni un solo movimiento en la base de datos.
	create view ambiciosos as
	SELECT u.nombre as nombre
	from USUARIO u
	where u.id_usuario not in(
	select u3.id_usuario
	from USUARIO u3 inner join TRANSACCION t
	on u3.id_usuario = t.id_usuario) and u.id_usuario in(
	select u2.id_usuario
	from USUARIO u2 inner join METAS_AHORRO ma
	on u2.id_usuario = ma.id_usuarioCreador

VISTA 3:

3.Vista donde aparece cuántos días han pasado desde que cada usuario recibió su última notificación.Muestra el nombre del usuario, la fecha de la última notificación y los días transcurridos. Si nunca han recibido una, usa IFNULL para poner "0".

```
-- 3.Vista donde aparece cuántos días han pasado desde que cada usuario recibió su última notificación.
-- Muestra el nombre del usuario, la fecha de la última notificación y
-- los días transcurridos. Si nunca han recibido una, usa IFNULL para poner "0".
create view tiempo_notificacion as
select u.nombre as nombre_usuario,
ifnull(max(r.fecha_notificacion),'no le ha llegado') as ultima_fecha,
```

```
ifnull(DATEDIFF(NOW(),max(r.fecha_notificacion)),'no tiene') as dias_diferencia
from USUARIO u left join RECIBE r
on u.id_usuario = r.id_usuario
group by u.id_usuario ;
```



9.PROCEDIMIENTOS,FUNCIONES Y TRIGGERS

9.1FUNCIONES

1.Función que determina cuál es la meta de ahorro más cercana a completarse para un usuario específico.

FUNCIÓN 1:

-- 1.Función que determina cuál es la meta de ahorro más cercana a completarse para un usuario específico.

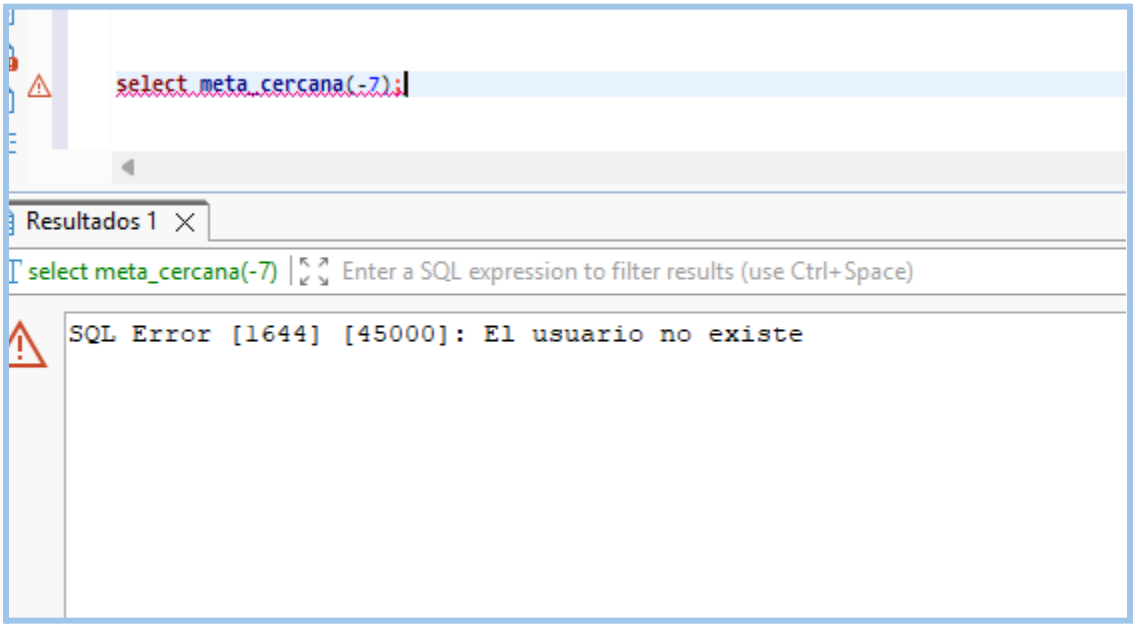
```
drop function if exists meta_cercana;
delimiter //
create function meta_cercana(id int)
returns int
deterministic
begin
	declare metaProx int;
	if exists (select 1 from USUARIO u where u.id_usuario=id) then
		select ma.id_meta
		into metaProx
		from METAS_AHORRO ma
		where (ma.cantidad_ahorrar-ma.cantidad_actual) =(
		select min((ma2.cantidad_ahorrar-ma2.cantidad_actual)) as resta
		from METAS_AHORRO ma2
		where ma2.estado != 'acabado' and ma2.id_usuarioCreador =id)
		and ma.id_usuarioCreador = id
		limit 1;

		if metaProx is null then
			set metaProx = -1;
		end if;
	else
		signal sqlstate '45000'
		set message_text='El usuario no existe';
	end if;
	return metaProx;
end //
delimiter ;
select meta_cercana(7);
```

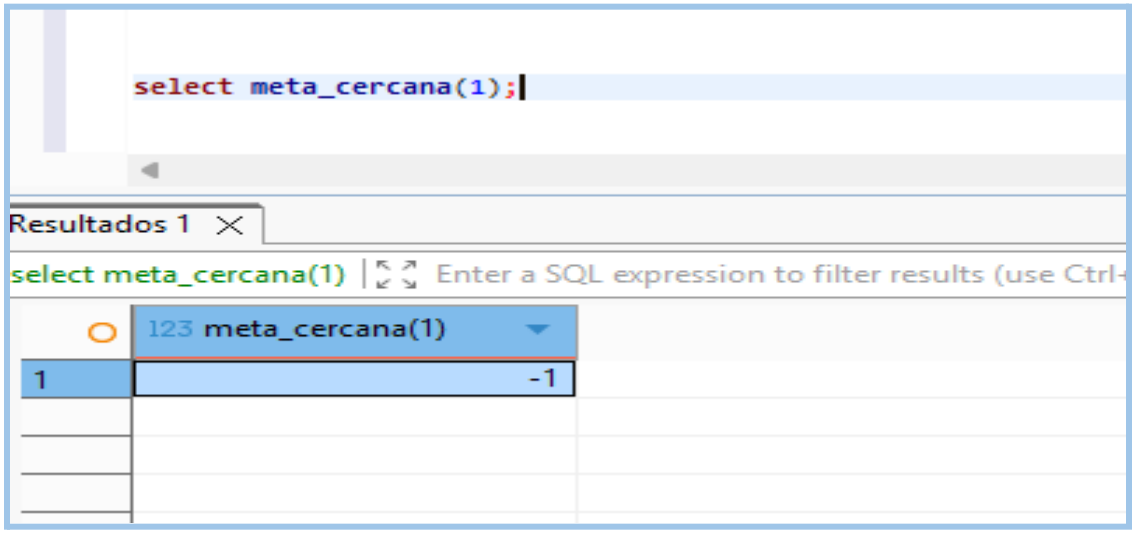
```
-- 1.Función que determina cuál es la meta de ahorro más cercana a completarse para un usuario específic

drop function if exists meta_cercana;

delimiter //
create function meta_cercana(id int)
returns int
deterministic
begin
    declare metaProx int;
    if exists (select 1 from USUARIO u where u.id_usuario=id) then
        select ma.id_meta
        into metaProx
        from METAS_AHORRO ma
        where (ma.cantidad_ahorrar-ma.cantidad_actual) =(
        select min((ma2.cantidad_ahorrar-ma2.cantidad_actual)) as resta
        from METAS_AHORRO ma2
        where ma2.estado != 'acabado' and ma2.id_usuarioCreador =id)
        and ma.id_usuarioCreador = id
        limit 1;

        if metaProx is null then
            set metaProx = -1;
        end if;
    else
        signal sqlstate '45000'
        set message_text='El usuario no existe';
    end if;
    return metaProx;
end //
delimiter ;

select meta_cercana(7);
```

esultados 1

lect meta_cercana(7) | Enter a SQL expression to filter results (use Ctrl+Space)

	123 meta_cercana(7)
1	7





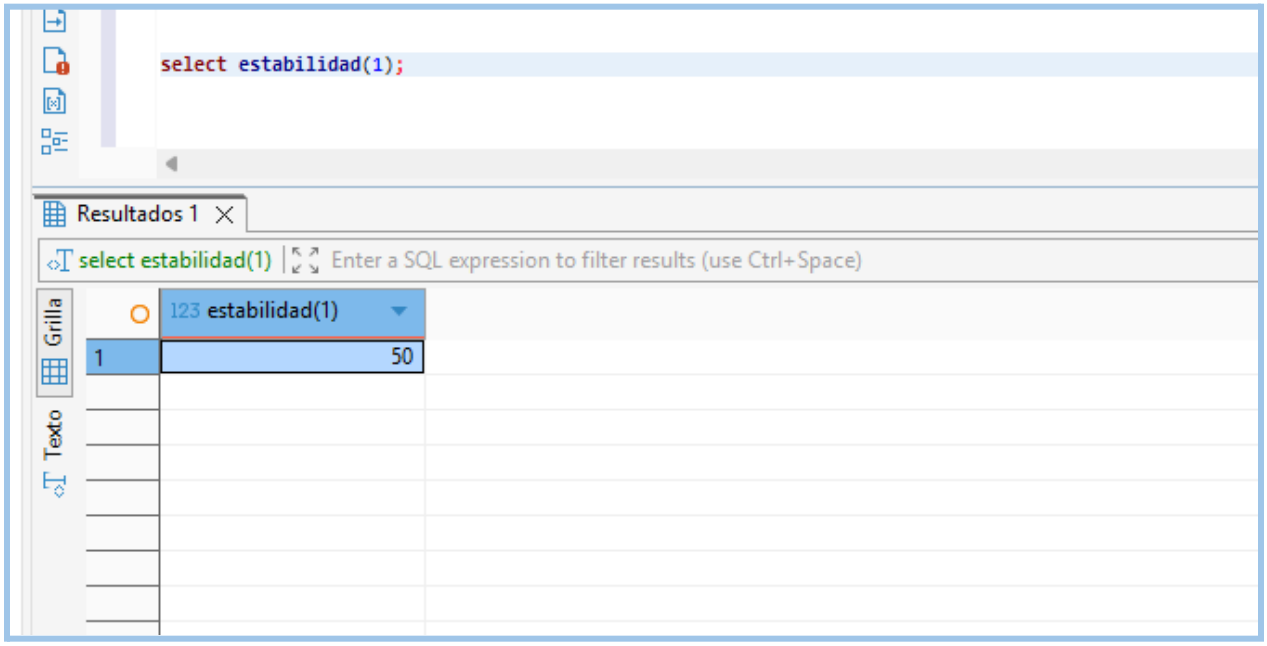
FUNCIÓN 2:

2. Se requiere una función que calcule el Índice de Estabilidad de un usuario basándose en su historial de movimientos.La función debe comparar el número de ingresos frente al número de retiradas totales.Un usuario estable es aquel que ingresa dinero con más frecuencia de la que lo retira.Si las retiradas superan en número a los

ingresos, el score baja.Si el usuario nunca ha retirado dinero, se le asigna la puntuación máxima.

```
-- 2. Se requiere una función que calcule el Índice de Estabilidad de un usuario basándose en su historial de movimientos.
-- La función debe comparar el número de ingresos frente al número de retiradas totales.
-- Un usuario estable es aquel que ingresa dinero con más frecuencia de la que lo retira.
-- Si las retiradas superan en número a los ingresos, el score baja.
-- Si el usuario nunca ha retirado dinero, se le asigna la puntuación máxima.
-- El resultado debe ser un valor del 1 al 100 que represente la solidez financiera del perfil.
drop function if exists estabilidad;
delimiter //
create function estabilidad(id int)
returns int
deterministic
begin
	declare mov int;
	declare ing int;
	declare puntos int;
	if exists (select 1 from USUARIO u where u.id_usuario=id) then
		SELECT count(*) as movimientos,
		(select count(*) from TRANSACCION t2 where t2.id_usuario =
t.id_usuario
		and t2.tipo like'ingreso')
		into mov, ing
		from TRANSACCION t
		where t.id_usuario = id;

		if(mov=ing) then
			set puntos = 100;
		elseif(ing=0 or mov=0) then
			set puntos=0;
		else
```

```
                    set puntos=(ing/mov)*100;
            end if;
        else
            signal sqlstate '45000'
            set message_text = 'Error, el usuario no existe';
        end if;
        return puntos;
        end //
delimiter ;
select estabilidad(8);
```



```
delimiter //
create function estabilidad(id int)
returns int
deterministic
begin
    declare mov int;
    declare ing int;
    declare puntos int;
    if exists (select 1 from USUARIO u where u.id_usuario=id) then
        SELECT count(*) as movimientos,
        (select count(*) from TRANSACCION t2 where t2.id_usuario = t.id_usuario
        and t2.tipo like'ingreso')
        into mov, ing
        from TRANSACCION t
        where t.id_usuario = id;

        if(mov=ing) then
            set puntos = 100;
        elseif(ing=0 or mov=0) then
            set puntos=0;
        else
            set puntos=(ing/mov)*100;
        end if;
    else
        signal sqlstate '45000'
        set message_text = 'Error, el usuario no existe';
    end if;
    return puntos;
    end //
delimiter ;

select estabilidad(8);
```

esultados 1

elect estabilidad(8) | Enter a SQL expression to filter results (use Ctrl+Space)

	123 estabilidad(8)
1	0

```
select estabilidad(-8);
```

esultados 1

elect estabilidad(-8) | Enter a SQL expression to filter results (use Ctrl+Space)

```
SQL Error [1644] [45000]: Error, el usuario no existe
```

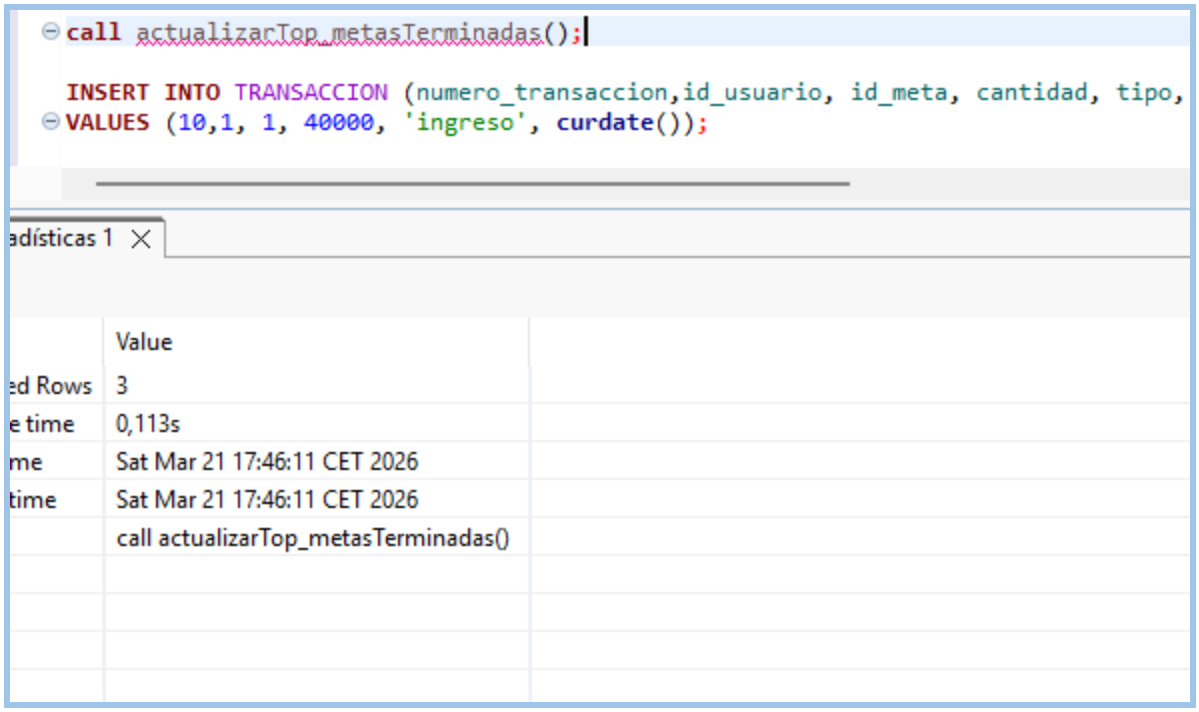
9.2PROCEDIMIENTOS

PROCEDIMIENTO 1:

1.Procedimiento que recalcula los 3 mejores usuarios del sistema. Cada vez que alguien termina una meta, el procedimiento limpia la tabla de posiciones y vuelve a insertar a los 3 usuarios que más metas han completado hasta el momento, ordenados de mayor a menor éxito

```
-- 1.Procedimiento que recalcula los 3 mejores usuarios del sistema. Cada vez que alguien, termina una meta, el procedimiento limpia la tabla de posiciones y vuelve a insertar a los 3 usuarios que más metas han completado hasta el momento, ordenados de mayor a menor éxito

drop TABLE if exists top_ganadores;
create table top_ganadores(
  id_usuario INT primary key,
  metas_acabadas INT NOT NULL
```

```
);
drop procedure if exists actualizarTop_metasTerminadas;
delimiter //
create procedure actualizarTop_metasTerminadas()
begin
        delete from top_ganadores;
        insert into top_ganadores(id_usuario,metas_acabadas)
        select uma.idUsuario, count(*) as metas_acanadas
        from usuarios_metasAcabadas uma
        group by uma.idUsuario
        order by count(*) desc
        limit 3;
end //
delimiter ;
```



	123 id_usuario	123 metas_acabadas
1	1	1
2	2	1
3	3	1

PROCEDIMIENTO 2:

2..Crea un procedimiento almacenado llamado sp_perfil_motivacional que actúe como un motor de comunicación personalizada.El procedimiento debe recibir el ID de un usuario y, haciendo uso de las funciones estabilidad() y meta_cercana(), generar un informe de estado

```
-- 2.Crea un procedimiento que actúe como un motor de comunicación personalizada.
-- El procedimiento debe recibir el ID de un usuario y, haciendo uso de las funciones
estabilidad() y meta_cercana(), generar un informe de estado
drop procedure if exists perfil_usuario;
delimiter //
create procedure perfil_usuario(in id int)
begin
	declare puntos int;
	declare metaProx int;
	declare mensaje varchar(100);
	declare metas varchar(100);
	if exists(select 1 from USUARIO u where u.id_usuario = id) then
		set puntos=estabilidad(id);
		set metaProx=meta_cercana(id);
```

```
            if(metaProx=-1) then
                set metas='no hay metas que terminar';
            else
                set metas=concat_ws(" ",'la meta más proxima que tienes que
acabar es ',metaProx);
            end if;
            if puntos >=70 then
                set mensaje=concat_ws(" ",'eres un ahorrador de elite,',metas);
            elseif puntos between 30 and 70 then
                set mensaje=concat_ws(" ",'var por buen camino,',metas);
            else
                set mensaje=concat_ws(" ",'necesitas mejorar,',metas);
            end if;
        else
            signal sqlstate '45000'
            set message_text='Error, el usuario no existe';
        end if;
        select mensaje;
end //
delimiter ;
call perfil_usuario(2);
```

```
-- 2.Crea un procedimiento almacenado llamado sp_perfil_motivacional que actúe como un motor de comunicación personalizada.
-- El procedimiento debe recibir el ID de un usuario y, haciendo uso de las funciones estabilidad() y meta_cercana(), generar un informe de estado
drop procedure if exists perfil_usuario;
delimiter //
create procedure perfil_usuario(in id int)
begin
    declare puntos int;
    declare metaProx int;
    declare mensaje varchar(100);
    declare metas varchar(100);
    if exists(select 1 from USUARIO u where u.id_usuario = id) then
        set puntos=estabilidad(id);
        set metaProx=meta_cercana(id);
        if(metaProx=-1) then
            set metas='no hay metas que terminar';
        else
            set metas=concat_ws(" ",'la meta más proxima que tienes que acabar es ',metaProx);
        end if;

        if puntos >=70 then
            set mensaje=concat_ws(" ",'eres un ahorrador de elite,',metas);
        elseif puntos between 30 and 70 then
            set mensaje=concat_ws(" ",'var por buen camino,',metas);
        else
            set mensaje=concat_ws(" ",'necesitas mejorar,',metas);
        end if;
    else
        signal sqlstate '45000'
        set message_text='Error, el usuario no existe';
    end if;
    select mensaje;
end //
delimiter ;

call perfil_usuario(2);
```

esultados 1 | Estadísticas 1

all perfil_usuario(2) | Enter a SQL expression to filter results (use Ctrl+Space)

	mensaje
1	var por buen camino no hay metas que terminar

```
        end if;
        if puntos >=70 then
            set mensaje=concat_ws(" ",'eres un ahorrador de elite,',metas);
        elseif puntos between 30 and 70 then
            set mensaje=concat_ws(" ",'var por buen camino,',metas);
        else
            set mensaje=concat_ws(" ",'necesitas mejorar,',metas);
        end if;
    else
        signal sqlstate '45000'
        set message_text='Error, el usuario no existe';
    end if;
    select mensaje;
end //
delimiter ;
call perfil_usuario(-2);
```

esultados 1

all perfil_usuario(-2) | Enter a SQL expression to filter results (use Ctrl+Space)

```
SQL Error [1644] [45000]: Error, el usuario no existe
```

PROCEDIMIENTO 3:

3. Crea un procedimiento donde se recorra la tabla usuario y para todos los que sean mayores de 30 le añadas un _1 a su nombre de usuario

```
-- 3.Crea un procedimiento donde se  recorrar la tabla usuario y para todos los que sean mayores de 30 le añadas un _1 a su nombre de usuario
drop procedure if exists dar_marca;
delimiter //
create procedure dar_marca()
begin
  declare idA int;
  declare consejos int;
```

```
  declare cursorF boolean default false;
  declare cursor_anio cursor for select u.id_usuario from USUARIO u;

   declare continue handler for not found set cursorF = true;

  open cursor_anio;
  fin: loop
     fetch cursor_anio into idA;

     if cursorF = true then
        leave fin;
     end if;

     update USUARIO u set u.nombre_usuario = concat(u.nombre_usuario,'_1') where
(datediff(now(),u.fecha_nacimiento)/365)>30 and u.id_usuario=idA;

  end loop fin;

  close cursor_anio;
end //
delimiter ;
call dar_marca();
```

```
-- 3.Crea un procedimiento donde recorrar la tabla usuario y para todos los que sean mayores de 30 le añadas un _1 a su nombre de usuario
drop procedure if exists dar_marca;
delimiter //
create procedure dar_marca()
begin
    declare idA int;
    declare consejos int;
    declare cursorF boolean default false;
    declare cursor_anio cursor for select u.id_usuario from USUARIO u;


    declare continue handler for not found set cursorF = true;

    open cursor_anio;
    fin: loop
        fetch cursor_anio into idA;

        if cursorF = true then
            leave fin;
        end if;

        update USUARIO u set u.nombre_usuario = concat(u.nombre_usuario,'_1') where (datediff(now(),u.fecha_nacimiento)/365)>30 and u.id_usuario=idA;

    end loop fin;

    close cursor_anio;
end //
delimiter ;

call dar_marca();


-- 2.Crea un trigger que realice un Control de seguridad para limitar a un máximo de 3 ediciones diarias por consejo y registro automático de cambios en
drop table if exists historial_cambios;
```

tadísticas 1

e	Value
ies	3
ted Rows	0
ute time	0,228s
time	0s
time	0,228s
time	2026-03-22 08:36:55.582
time	2026-03-22 08:36:55.811

	id_usuario	nombre	apellidos	fecha_nacimiento	dni	nombre_usuario	contrasenia	pais	correo
94	94	Lainey	Corwin	1901-07-11	522-76-0086	lcorwin2l_1	kY0'c(Cp~|x.0	Peru	lcorwin2l@stanford.edu
95	95	Lynda	Cafe	1950-07-31	397-86-7580	lcafe2m_1	dU4\fHJOZfMxk/	Portugal	lcafe2m@photobucket.com
96	96	Gena	Boundy	1959-09-10	888-34-7412	gboundy2n_1	lS4+ITmCPBuQpF'	Russia	gboundy2n@histats.com
97	97	Amalia	Camock	1914-02-07	267-99-1515	acamock2o_1	qQ9}jua#b$3NK	China	acamock2o@seesaa.net
98	98	Ruth	Srawley	1916-03-27	597-72-5409	rsrawley2p_1	oD2/F&mMAM	China	rsrawley2p@bravesites.com
99	99	Barbabas	Cricket	1988-12-11	780-29-8237	bcricket2q_1	yJ0&tc2Hq{$+Iq"	China	bcricket2q@marriott.com
100	100	Lowell	MacGaughy	1965-03-05	882-29-4225	lmacgaughy2r_1	hT7+o|~0t3n=	Saint Vincent an	lmacgaughy2r@hc360.com
101	101	Mortie	Lotwich	1937-09-27	480-58-8097	mlotwich2s_1	fA8#4RRcoT}T	France	mlotwich2s@kickstarter.com
102	102	Adolf	Tyt	1975-01-07	781-66-0964	atyt2t_1	fV4)ZHf)OmuY	China	atyt2t@technorati.com
103	103	Marcia	Kears	1986-10-15	215-21-9293	mkears2u_1	yA2%MljX'@#pZ	Portugal	mkears2u@joomla.org
104	104	Sherri	Danev	1934-09-08	360-60-7141	sdanev2v_1	oA4"3nRbz_PF]NI?	China	sdanev2v@sciencedirect.com
105	105	Regan	Niblock	1979-04-18	628-49-4474	rniblock2w_1	kD7&@&8"bA24a	China	rniblock2w@cisco.com
106	106	Dorolice	Scotson	1985-03-26	499-45-4420	dscotson2x_1	qK9}cj!+$Ib	France	dscotson2x@fc2.com
107	107	Alfy	McNelis	1920-02-23	449-71-5771	amcnelis2y_1	fQ5&+2N@i2YyrRs	China	amcnelis2y@patch.com
108	108	Loni	Ludman	1945-12-27	613-31-1297	lludman2z_1	oU2?n7oY(W1	Indonesia	lludman2z@over-blog.com
109	109	Ellyn	Gatsby	1911-12-30	275-29-7656	egatsby30_1	qA2@tLdJ/<PcYO	China	egatsby30@usda.gov
110	110	Diane	Mayell	1978-10-05	625-63-8620	dmayell31_1	tJ4/zH9msd	Philippines	dmayell31@trellian.com
111	111	Quintina	Sallowaye	1936-05-29	350-19-6816	qsallowaye32_1	fB8$v(g0wU_	Vietnam	qsallowaye32@utexas.edu
112	112	Enrichetta	Redgate	1972-02-13	194-31-5409	eredgate33_1	kZ9&/A`+,	Paraguay	eredgate33@umich.edu
113	113	Zebadiah	Davisson	1964-12-13	817-64-9712	zdavisson34_1	sD4|vmP99c)9>mUo	China	zdavisson34@cbc.ca
114	114	Beverie	Winsom	1931-05-16	542-11-6944	bwinsom35_1	eY2{6N+8bz*9yc	Tanzania	bwinsom35@irs.gov
115	115	Brigham	Hagston	1913-04-21	519-89-5097	bhagston36_1	vU6,%)16Ww)~	China	bhagston36@eventbrite.com
116	116	Madelyn	Izak	1926-12-15	791-47-1611	mizak37_1	zL4$(chB	Indonesia	mizak37@google.de
117	117	Jaimie	Meggison	1970-05-23	562-34-9545	jmeggison38_1	jF0(&l=}	Cuba	jmeggison38@wisc.edu
118	118	Bernita	Cammis	1933-11-13	119-74-3315	bcammis39_1	bU9/?K?J)8#i	China	bcammis39@yelp.com
119	119	Ivette	Carnoghan	1949-10-08	361-08-3808	icarnoghan3a_1	lH5!dKsFXUPY%`/o	Azerbaijan	icarnoghan3a@virginia.edu
120	120	Oralee	Kenwin	1923-01-27	855-65-0551	okenwin3b_1	qP4~vx9i	Indonesia	okenwin3b@berkeley.edu

9.3TRIGGERS

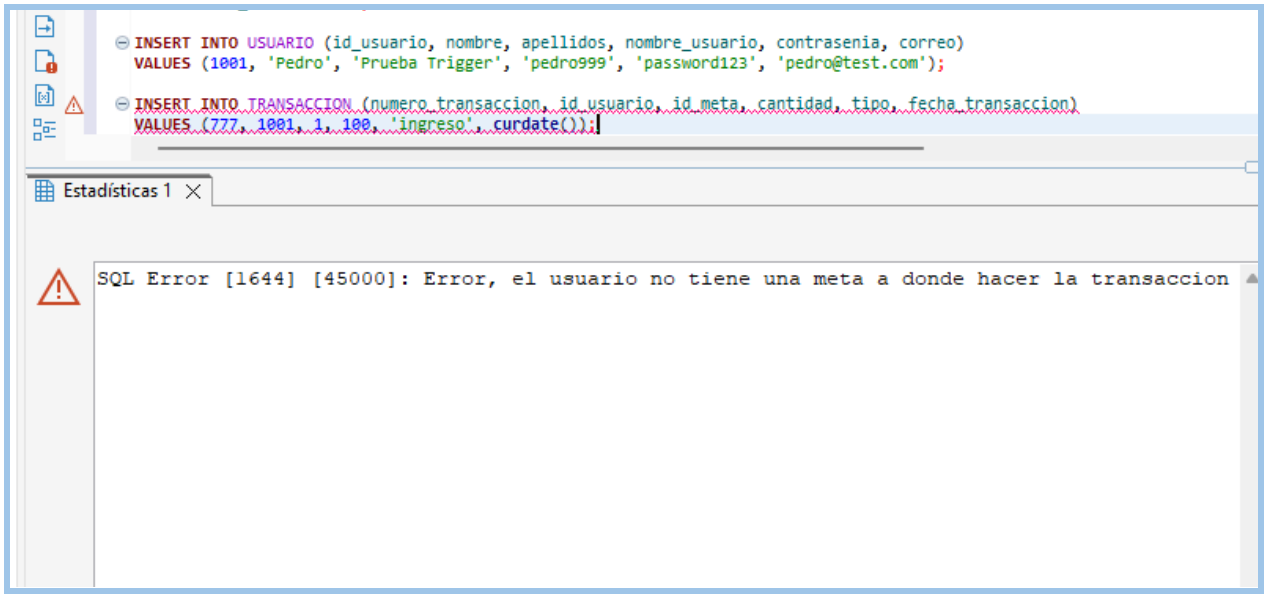
TRIGGER 1:

1.Trigger que, tras registrar un ingreso, comprueba si se ha alcanzado el objetivo de ahorro.Si se alcanza, inserta el registro en la tabla de metas finalizadas, marca la meta como 'acabada' y actualiza el ranking de los mejores ahorradores.

```
–Trigger que, tras registrar un ingreso, comprueba si se ha alcanzado el objetivo de ahorro.
-- Si se alcanza, inserta el registro en la tabla de metas finalizadas, marca la meta como 'acabada' y actualiza el ranking de los mejores ahorradores.
create table usuarios_metasAcabadas(
id_logro int AUTO_INCREMENT PRIMARY KEY,
idUsuario int unsigned not null,
idMeta int unsigned unique not null,
cantidadF decimal(15,2) not null
) ;
drop trigger if exists metas_acabadas;
delimiter //
```

```
create trigger metas_acabadas
after insert on TRANSACCION
for each row
begin
	declare cantidadActual decimal(15,2);
	declare cantidadAhorrar decimal(15,2);
	if exists(select 1 from METAS_AHORRO ma where
ma.id_usuarioCreador=new.id_usuario) then
		select ma.cantidad_actual,ma.cantidad_ahorrar
			into cantidadActual, cantidadAhorrar
			from METAS_AHORRO ma
			where ma.id_meta=new.id_meta;

		if new.tipo like 'ingreso' then
			update METAS_AHORRO ma set ma.cantidad_actual =
ma.cantidad_actual +new.cantidad
			where ma.id_meta =new.id_meta;
			if cantidadAhorrar <=(cantidadActual + new.cantidad) then
				insert into
usuarios_metasAcabadas(idUsuario,idMeta,cantidadF)
values(new.id_usuario,new.id_meta,cantidadActual+new.cantidad);
				update METAS_AHORRO ma set ma.estado = 'acabado'
where ma.id_meta = new.id_meta;
				call actualizarTop_metasTerminadas();
			end if;
		else
			if cantidadActual-new.cantidad>0 then
				update METAS_AHORRO ma set ma.cantidad_actual =
ma.cantidad_actual -new.cantidad
				where ma.id_meta=new.id_meta;
			else
				signal sqlstate '45000'
				set message_text='Error, estás retirando más de lo que
puedes';
			end if;
```

```
            end if;
        else
            signal sqlstate '45000'
            set message_text='Error, el usuario no tiene una meta a donde hacer la
transaccion';
        end if;
end //
delimiter ;
```



```
delimiter //
create trigger metas_acabadas
after insert on TRANSACCION
for each row
begin
    declare cantidadActual decimal(15,2);
    declare cantidadAhorrar decimal(15,2);
    if exists(select 1 from METAS_AHORRO ma where ma.id_usuarioCreador=new.id_usuario) then
        select ma.cantidad_actual,ma.cantidad_ahorrar
            into cantidadActual, cantidadAhorrar
            from METAS_AHORRO ma
            where ma.id_meta=new.id_meta;

        if new.tipo like 'ingreso' then
            update METAS_AHORRO ma set ma.cantidad_actual = ma.cantidad_actual +new.cantidad
            where ma.id_meta =new.id_meta;
            if cantidadAhorrar <=(cantidadActual + new.cantidad) then
                insert into usuarios_metasAcabadas(idUsuario,idMeta,cantidadF) values(new.id_usuario,new.id_meta,cantidad
                update METAS_AHORRO ma set ma.estado = 'acabado' where ma.id_meta = new.id_meta;
                call actualizarTop_metasTerminadas();
            end if;
        else
            if cantidadActual-new.cantidad>0 then
                update METAS_AHORRO ma set ma.cantidad_actual = ma.cantidad_actual -new.cantidad
                where ma.id_meta=new.id_meta;
            else
                signal sqlstate '45000'
                set message_text='Error, estás retirando más de lo que puedes';
            end if;
        end if;
    else
        signal sqlstate '45000'
        set message_text='Error, el usuario no tiene una meta a donde hacer la transaccion';
    end if;
end //
delimiter ;
```

Estadísticas 1

Name	Value
Queries	3
Updated Rows	0
Execute time	0,277s
Fetch time	0s
Total time	0,277s
Start time	2026-03-23 16:11:57.215
Finish time	2026-03-23 16:11:57.492

```
INSERT INTO TRANSACCION (numero_transaccion,id_usuario, id_meta, cantidad, tipo, fecha_transaccion)
VALUES (10,1, 1, 40000, 'ingreso', curdate());

INSERT INTO TRANSACCION (numero_transaccion,id_usuario, id_meta, cantidad, tipo, fecha_transaccion)
VALUES (9,2, 2, 80000, 'ingreso', curdate());

INSERT INTO TRANSACCION (numero_transaccion,id_usuario, id_meta, cantidad, tipo, fecha_transaccion)
VALUES (4,3, 3, 30000, 'ingreso', curdate());

select*from top_ganadores tg;

show columns from TRANSACCION;
```

top_ganadores 1

select*from top_ganadores tg | Enter a SQL expression to filter results (use Ctrl+Space)

	123 id_usuario	123 metas_acabadas
1	1	1
2	2	1
3	3	1

```
select*from top_ganadores tg;

select*
from METAS_AHORRO ma
where ma.estado like 'acabado';
```

METAS_AHORRO 1

elect* from METAS_AHORRO ma where ma.estado like 'acabado' | Enter a SQL expression to filter results (use Ctrl+Space)

	123 id_meta	123 cantidad_ahorrar	A-Z nombre	123 cantidad_actual	A-Z estado	A-Z prioridad	123 id_usuarioCreador
1	1	37.561,44	Ahorro salud	40.541,75	acabado	media	1
2	2	73.966,42	Ahorro ropa	80.596,59	acabado	alta	2
3	3	27.756,76	Ahorro imprevistos	30.000	acabado	media	3

```
INSERT INTO TRANSACCION (numero_transaccion,id_usuario, id_meta, cantidad, tipo, fecha_transaccion)
VALUES (6,4, 4, 30000, 'retirada', curdate());
```

ETAS_AHORRO 1

ISERT INTO TRANSACCION (numero_transaccion,id_usuario, id_meta, Enter a SQL expression to filter results (use Ctrl+Space)

```
SQL Error [1644] [45000]: Error, estás retirando más de lo que puedes
```

TRIGGER 2:

2.Crea un trigger que realice un Control de seguridad para limitar a un máximo de 3 ediciones diarias por consejo y registro automático de cambios en tabla de histórico.

```
-- 2.Crea un trigger que realice un Control de seguridad para limitar a un máximo de 3 ediciones diarias por consejo y registro automático de cambios en tabla de histórico.
drop table if exists historial_cambios;
create table historial_cambios(
	idHistorial int auto_increment primary key,
	idUsuario int,
	idConsejo int,
	contenido varchar(250),
	fecha date
);
drop trigger if exists historial_consejos;
delimiter //
create trigger historial_consejos
before update on CONSEJO
for each row
begin
```

```
        declare numeroCambios int;
        if exists (select 1 from USUARIO u where
u.id_usuario=new.id_usuarioPublica) then
                if old.id_usuarioPublica=new.id_usuarioPublica then
                        select count(*)
                        into numeroCambios
                        from historial_cambios ca
                        where ca.idUsuario=old.id_usuarioPublica;

                        if numeroCambios<3 then
                                if(new.contenido!=old.contenido) then
                                        insert into
historial_cambios(idUsuario,idConsejo,contenido,fecha)

values(old.id_usuarioPublica,old.id_consejo,old.contenido,curdate());
                                else
                                        signal sqlstate '45000'
                                        set message_text='El contenido que deseas
introducir es igual al anteior';
                                end if;
                        else
                                signal sqlstate '46000'
                                set message_text='No se puede actualizar más veces';
                        end if;
                else
                        signal sqlstate '45000'
                        set message_text='Error, el usuario no ha publicado un consejo
antes';
                end if;
        else
                signal sqlstate '47000'
                set message_text='El usuario no puede actualizar un mensaje que no ha
publicado';
        end if;
end //
```

```
delimiter ;


select*from historial_cambios;
```

```
drop trigger if exists historial_consejos;
delimiter //
create trigger historial_consejos
before update on CONSEJO
for each row
begin
    declare numeroCambios int;
    if exists (select 1 from USUARIO u where u.id_usuario=new.id_usuarioPublica) then
        if old.id_usuarioPublica=new.id_usuarioPublica then
            select count(*)
            into numeroCambios
            from historial_cambios ca
            where ca.idUsuario=old.id_usuarioPublica;

            if numeroCambios<3 then
                if(new.contenido!=old.contenido) then
                    insert into historial_cambios(idUsuario,idConsejo,contenido,fecha)
                    values(old.id_usuarioPublica,old.id_consejo,old.contenido,curdate());
                else
                    signal sqlstate '45000'
                    set message_text='El contenido que deseas introducir es igual al anteior';
                end if;
            else
                signal sqlstate '46000'
                set message_text='No se puede actualizar más veces';
            end if;
        else
            signal sqlstate '45000'
            set message_text='Error, el usuario no ha publicado un consejo antes';
        end if;
    else
        signal sqlstate '47000'
        set message_text='El usuario no puede actualizar un mensaje que no ha publicado';
    end if;
end //
delimiter ;

update CONSEJO c set c.contenido='Prueba del error'
```

tadísticas 1

e	Value
es	3
ted Rows	0
ıte time	0,226s
time	0s
time	0,226s
time	2026-03-23 16:38:34.091
ı time	2026-03-23 16:38:34.318

```
update CONSEJO c set c.contenido ='Recuerda reutilizar lo que aun sirva' where c.id_usuarioPublica = 13 and c.id_consejo ='13';
select*from historial_cambios;
```

*from historial_cambios | Enter a SQL expression to filter results (use Ctrl+Space)

123 idHistorial	123 idUsuario	123 idConsejo	A-Z contenido	fecha
1	2	2	Cuidarte y organizar tu tiempo también es una forma de avanzar. Pequeñas acciones diarias pueden mejorar mucho tu bienestar.	2026-03-22
2	2	502	Schreiben Sie alle täglichen Ausgaben auf, um unnötige Kosten zu identifizieren und Geld zu sparen.	2026-03-22
3	2	502	Recuerda reutilizar lo que aun sirva	2026-03-22
4	13	13	Equivocarse es parte del camino. Lo importante es reconocer los errores, aprender de ellos y continuar con más confianza.	2026-03-22

```
update CONSEJO c set c.contenido ='Recuerda reutilizar lo que aun sirva' where c.id_usuarioPublica = 13 and c.id_consejo ='13';
select*from historial_cambios;
```

orial_cambios 1

date CONSEJO c set c.contenido ='Recuerda reutilizar lo que aun sirv | Enter a SQL expression to filter results (use Ctrl+Space)

```
SQL Error [1644] [45000]: El contenido que deseas introducir es igual al anteior
```

```
update CONSEJO c set c.contenido ='Recuerda reutilizar lo que aun sirva' where c.id_usuarioPublica = 13 and c.id_consejo ='13';
update CONSEJO c set c.contenido ='Recuerda reutilizar' where c.id_usuarioPublica = 13 and c.id_consejo ='13';
update CONSEJO c set c.contenido ='Aprovecha lo que tienes en casa' where c.id_usuarioPublica = 13 and c.id_consejo ='13';
update CONSEJO c set c.contenido ='Recuerda reutilizar' where c.id_usuarioPublica = 13 and c.id_consejo ='13';
update CONSEJO c set c.contenido ='No malgastes' where c.id_usuarioPublica = 13 and c.id_consejo ='13';
select*from historial_cambios;
```

storial_cambios 1

pdate CONSEJO c set c.contenido ='No malgastes' where c.id_usuario | Enter a SQL expression to filter results (use Ctrl+Space)

```
SQL Error [1644] [46000]: No se puede actualizar más veces
```

```
UPDATE CONSEJO SET id_usuarioPublica = 888 WHERE id_consejo = 1;
```

Estadísticas 1

```
SQL Error [1644] [45000]: Error, el usuario no ha publicado un consejo antes
```

```
UPDATE CONSEJO SET id_usuarioPublica = 8888 WHERE id_consejo = 1;
```

Estadísticas 1

```
SQL Error [1644] [47000]: El usuario no puede actualizar un mensaje que no ha publicado
```

10.SUBIDA A GITHUB

Una vez finalicé mi proyecto, realicé la subida de los archivos a github para tenerlos guardados en un repositorio.

Enlace a GitHub: https://github.com/sergioruiz28282/Ahorros_ProyectoBD_SRO.git

11.CONCLUSIÓN

Con este proyecto he aprendido la importancia que tienen las funciones,los procedimientos y los triggers dentro de una base de datos permitiendo ver que las tablas no son simples almacenes de datos, sino que puede realizar acciones propias.Gracias a los triggers he logrado que la base de datos se proteja sola y vigile los cambios de los usuarios sin que yo tenga que estar encima, mientras que con las funciones y los procedimientos con cursores he podido automatizar cálculos y tareas pesadas de forma mucho más rápida y limpia.

Punto de enlace:proyectoahorros.c7c2qiumu8ih.us-east-1.rds.amazonaws.com